SESSION 1 · NOSQL DATABASES · 9:00 – 13:45

NoSQL: when tables are not enough

Don't panic, SQL is not dead. It is just tired.

Badmavasan KIROUCHENASSAMY

"Zwipe" ice-breaker · Tools vs technologies · MongoDB from A to `$lookup`

BEFORE WE START

You need Docker

Install

- **Windows / macOS**: Docker Desktop (docs.docker.com/get-docker)
- **Linux**: Docker Engine + the Compose plugin (`docker-compose-plugin`)
- One laptop with Docker **per group** is enough for the hands-on part

Check

```
docker --version
docker compose version
docker pull mongo:7.0      # download the image now
```

If all three commands work, you are ready for 10:50.

Until then: **pen and paper**. Laptops stay closed during the ice-breaker.

SESSION PLAN

9:00 → 13:45: 3 hours of class, then the lab assignment

Time	What	Mode
9:00	Welcome, briefing for the Zwipe ice-breaker	Plenary
9:10	Ice-breaker: design Zwipe in SQL (20 min, pen and paper)	Groups of 3-4
9:30	Presentations and cross-critiques	Groups
9:50	Debrief: where SQL breaks, the CAP theorem	Plenary
10:05	Tools vs technologies, what NoSQL is	Theory
10:20	Break	☕
10:30	MongoDB: cluster, server, database, collection, document	Theory
10:50	Hands-on: find, filters, insert, update, delete	Practice, laptops
11:35	When to choose Mongo, when definitely not	Theory + debate
11:50	Quiz and session 2 teaser	Game
12:00	**Graded assignment**: the New York restaurants dataset (`tp-1/TP.md`)	Self-paced, pairs
13:45	End, assignment hand-in	

01 / ICE-BREAKER

Zwipe

A short-video app. You are the data team. All you have is SQL.

CONTEXT

Zwipe, the app that blew up this summer

- Videos from 6 to 180 seconds, infinite "For you" feed
- Launched in June 2026, during the World Cup
- **50 million users** in 4 months
- Lives, duets, polls, sounds, affiliate products…
- A new feature every week

Your mission

The team has decided: **everything in SQL** (PostgreSQL). Propose the schema and the queries that will hold the load.

Any resemblance to an existing app is… you know which one.

THE 5 USE CASES TO COVER

Every group handles all five

	Use case	Constraint
U1	**"For you" feed**: the next 20 videos + a view counter that moves live	< 50 ms, 50 M scrolls / min
U2	**Video sheet**: sound, duet, filter, subtitles, location, product, poll… and a new option every week	No two videos with the same fields
U3	**"Your friends liked"**: recommendations via friends of friends of friends	3 levels, real time
U4	**Analytics**: view, like, skip, replay, aggregated per hour	2 M events / second
U5	**Live**: chat + gifts + top donors leaderboard	300,000 viewers, leaderboard refreshed 10×/s

INSTRUCTIONS

20 minutes, then we present

To produce

- The **schema**: tables, keys, indexes
- **One SQL query** per use case (pseudo-SQL accepted)
- One weak point you own up to

Materials

- **Pen and paper**, whiteboard
- No laptop, no phone, no ChatGPT
- **3 minutes** of presentation per group, sheet in hand

WHILE YOU WORK

Timer

20:00

Press `t` to start / pause, `r` to reset to 20:00

PRESENTATIONS

Listening grid for the other groups

While a group presents, the others note: **"where does it break?"**

Questions to ask

- How many **JOINs** to render one screen?
- What happens when product adds a field on **Friday at 6 pm**?
- How many **servers**? Which one receives the writes?

And also

- How many rows for **friends of friends of friends**?
- What does **2 M INSERT / s** cost on an indexed table?
- The live leaderboard: **ORDER BY** over how many rows, how many times?

02 / DEBRIEF

Where SQL breaks

Your schemas were good. The problem is the context.

THE SCHEMA (ALMOST) ALL OF US DREW

Version 1, Monday morning

```
CREATE TABLE users   (id, handle, country, age, bio);
CREATE TABLE videos  (id, author_id, title, duration, views, likes);
CREATE TABLE follows (follower_id, followed_id);
CREATE TABLE likes   (user_id, video_id, ts);
CREATE TABLE events  (id, type, user_id, video_id, ts);
CREATE TABLE live_messages (id, live_id, user_id, text, amount, ts);
```

Clean. Normalized. Your relational databases teacher is proud.

THE SCHEMA (ALMOST) ALL OF US DREW

Version 7, Friday 6 pm

```
CREATE TABLE users   (id, handle, country, age, bio, premium_since, premium_plan);
CREATE TABLE videos  (id, author_id, title, duration, views, likes,
                      duet_with, duet_type, place_name, place_lat, place_lng,
                      poll_question, filter, pinned, product_id, ...);
CREATE TABLE video_subtitles (video_id, language);
CREATE TABLE poll_options    (video_id, text, votes);
CREATE TABLE products (id, name, price);
CREATE TABLE follows (follower_id, followed_id);
CREATE TABLE likes   (user_id, video_id, ts);
CREATE TABLE events  (id, type, user_id, video_id, ts, watch_time, device);
CREATE TABLE live_messages (id, live_id, user_id, text, amount, ts);
-- ALTER TABLE videos ADD COLUMN ... (x14 this week)
```

U1 · "FOR YOU" FEED

One scroll = one join. 50 M scrolls / min.

Scenario. Léa opens Zwipe. The app must show the next 20 videos of her feed **immediately**, with the author's handle and the like count, and every view increments the view counter. 50 million people do the same every minute.

```sql
SELECT v.*, u.handle, COUNT(l.user_id) AS likes
FROM videos v
JOIN users u  ON u.id = v.author_id
LEFT JOIN likes l ON l.video_id = v.id
WHERE v.id IN (/* 20 ids from the recommender */)
GROUP BY v.id, u.handle;

UPDATE videos SET views = views + 1 WHERE id = 42;
-- ×100,000 / second on the same row
```

- The join is fine… once. Not 800,000 times per second.
- The view counter: **a lock on one row**, everybody queues up.
- You said "we'll add a cache"? Great. **Which cache?**

Hint: a **key → value** structure, in memory, that can do `INCR` in 1 µs.

40 columns, 35 are NULL

Scenario. Product ships one option a week. This video has a **poll**, that one a **duet** and a **location**, the third an affiliate **product** and subtitles in 4 languages. No two have the same fields, and the full sheet must render in a single read.

```
-- option A: everything in videos
ALTER TABLE videos ADD COLUMN poll_question TEXT;
ALTER TABLE videos ADD COLUMN duet_with INT;
ALTER TABLE videos ADD COLUMN duet_type VARCHAR(10);
-- … 14 ALTERs later, 35 NULL columns per row

-- option B: EAV (Entity-Attribute-Value)
CREATE TABLE video_attrs (video_id, key, value TEXT);
-- congratulations, you reinvented JSON, but worse
```

- Every video has a **different profile**: duet, poll, location, product…
- `ALTER TABLE` on 2 billion rows: you plan it, you don't "push" it.
- EAV: no more typing, no useful indexes, `JOIN`s to read a single video.

Hint: what if the video were stored **the way it is read**, one object with whatever fields it has?

U3 · "YOUR FRIENDS LIKED"

Friends of friends of friends = 3 self-joins

Scenario. Under a video, the "Your friends liked" block. For Léa, we must go to her follows, to their follows, then one more level, and find the videos liked by those **3 levels**, in real time.

```sql
SELECT DISTINCT l.video_id
FROM follows f1
JOIN follows f2 ON f2.follower_id = f1.followed_id
JOIN follows f3 ON f3.follower_id = f2.followed_id
JOIN likes l    ON l.user_id    = f3.followed_id
WHERE f1.follower_id = 42;
-- 300 follows × 300 × 300 = 27,000,000 intermediate rows
-- for ONE user. Add a level: 8 billion.
```

- Relational is good at **fixed relations**, not at **traversing** a network.
- Every level multiplies. The execution plan looks like a scream.
- Recursive CTEs exist. They don't scale any better.

Hint: a model where "follows" is a **physical link** between two nodes, traversed in O(neighbours).

2 million INSERTs per second

Scenario. Every view, like, skip and replay is an event: **2 million per second**. The internal dashboard wants, per hour and per type, the number of events over the last 24 h.

```sql
INSERT INTO events (type, user_id, video_id, ts)
VALUES ('view', 42, 1337, now());
-- × 2,000,000 / s, a single write server,
-- 4 B-tree indexes to maintain on every row

SELECT date_trunc('hour', ts), type, COUNT(*)
FROM events WHERE ts > now() - interval '1 day'
GROUP BY 1, 2;
-- scan of 170 billion rows. Good night.
```

- Classic SQL writes to **one primary**. One. Replicas only serve reads.
- B-tree indexes love reads and **hate massive writes**.
- Aggregating per hour reads 100 % of the columns when we want 2.

Hint: **column-oriented** storage, spread over N nodes that all write, sorted by time.

Top 100 donors, refreshed 10 times per second

Scenario. A live with **300,000 viewers**. The chat scrolls, (paid) gifts drop in, and the leaderboard of the 100 biggest donors is shown on screen, refreshed 10 times per second.

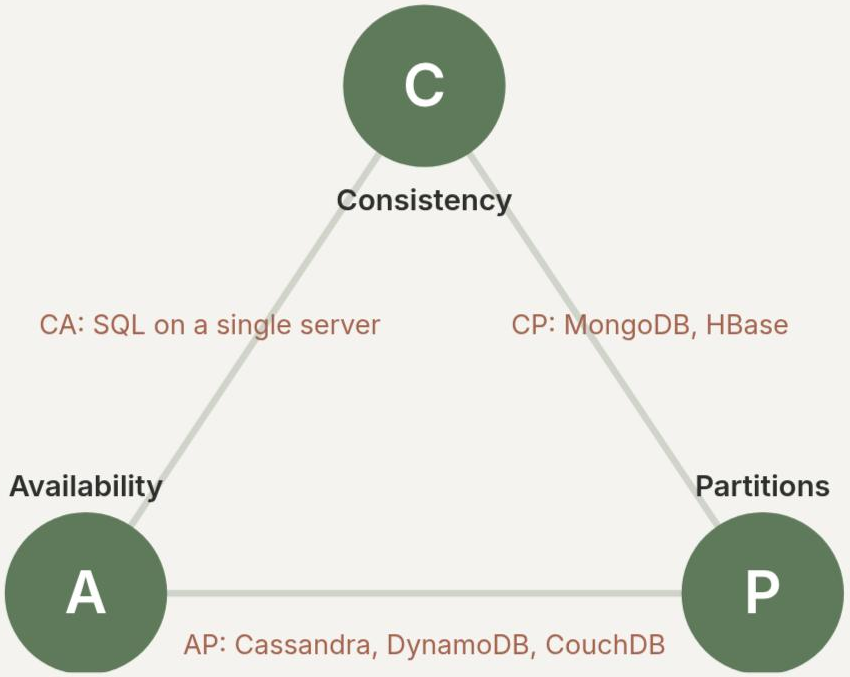
```sql
SELECT user_id, SUM(amount) AS total
FROM live_messages
WHERE live_id = 7 AND amount > 0
GROUP BY user_id
ORDER BY total DESC
LIMIT 100;
-- 300,000 viewers, 10×/s = 3 M aggregations/s
-- plus the chat writing 50,000 messages/s next to it
```

- A leaderboard is a **maintained sorted structure**, not a replayed query.
- The chat doesn't need durability: it needs **broadcast** (pub/sub).
- Gifts, on the other hand, must be durable and transactional. Two needs, two tools.

Hint: an in-memory **sorted set** + a publish channel. And a SQL database for the money.

THE THEOREM EVERYONE QUOTES

CAP: three promises, you only keep two



	The promise	To get it
C	Everyone reads **the same value** at the same time	Your balance is identical at the ATM in Lille and the one in Nice
A	Every request gets **an answer**, even a slightly stale one	The ATM in Nice always answers, even if head office is unreachable
P	The system keeps going when the **network between servers** fails	The Lille–Nice cable is cut, both branches stay open

As soon as you have **several servers**, the network will eventually fail: P is imposed. The real choice is C or A during the outage.

CAP IN PRACTICE

The cable is cut. What do we do?

Scenario. Zwipe has one server in Paris and one in Frankfurt. The link between them goes down. Léa likes a video on Paris. At the same instant, Tom asks Frankfurt for that video's like count.

Choose C: consistency

Frankfurt **refuses to answer** until it is sure it has the latest value. Tom sees an error or a spinner.

Tools: MongoDB (default settings), HBase, SQL with synchronous replication.

Good for: gift money, stock levels, a balance, a booking.

Choose A: availability

Frankfurt **answers with what it has**, maybe 2 seconds old. Tom sees 1,041 likes instead of 1,042. We reconcile when the cable is back.

Tools: Cassandra, DynamoDB, CouchDB, DNS.

Good for: view counters, feed, chat, shopping cart.

Classic SQL on **a single server** doesn't have this problem: no network between nodes, so C and A at once. It isn't bad, it is **optimised for something else**: strong consistency, transactions, stable relations. But a single server doesn't scale.

RECAP

Each use case has its NoSQL family

Case	SQL pain	NoSQL family	Example tool
U1 Feed	Repeated joins, lock on a counter	In-memory **key-value**	Redis, Memcached
U2 Video sheet	NULL columns, ALTER TABLE, EAV	**Document**	MongoDB, CouchDB
U3 Friends of friends	Exploding self-joins	**Graph**	Neo4j, Neptune
U4 Analytics	Massive writes, single primary	**Wide-column / time series**	Cassandra, ClickHouse, InfluxDB
U5 Live	Replayed ORDER BY, no broadcast	**Key-value** + pub/sub	Redis (sorted sets, streams)

The gift money (U5), though, stays in SQL: transactions and strong consistency.

03 / VOCABULARY

Tools vs technologies

"We'll use MongoDB" is not an architecture decision.

THE SLIDE EVERYONE GETS CONFUSED ABOUT

A technology is a concept. A tool is a product.

Technology (the what)	Tools (the with what)
JSON document storage	MongoDB, CouchDB, Firestore, Amazon DocumentDB
Key-value	Redis, Memcached, DynamoDB, etcd
Graph	Neo4j, Amazon Neptune, ArangoDB
Wide-column	Cassandra, ScyllaDB, HBase
Time series	InfluxDB, TimescaleDB, ClickHouse (columnar analytics)
Search engine	Elasticsearch, OpenSearch, Meilisearch
Vectors (AI, similarity)	Qdrant, Pinecone, pgvector, Milvus
Relational	PostgreSQL, MySQL, SQLite, Oracle

Choose the technology first (the data model), then the tool (licence, hosting, team, price).

MINI-QUIZ

Tool or technology?

- PostgreSQL → tool (relational)
- SQL → neither: a **language**
- JSONB in Postgres → a **technology** (document) inside a relational tool. Boundaries move.
- Kafka → tool, technology "distributed event log". Not a database.
- Firebase → a **platform** that contains a document tool (Firestore)
- "The cloud" → someone else's computer

DEFINITION

NoSQL = Not only SQL

- Not "no SQL": **not only** SQL. The word comes from a meetup hashtag in San Francisco, June 2009.
- What the families share: you **give up** something from the relational model to **gain** something else.

You give up	You gain	Zwipe example
An **enforced schema**	Flexibility	The video with a poll and the one with a duet live in the same collection, no `ALTER TABLE`
Joins	Fast local reads	The video sheet (author, sound, poll) is read as **one** document
Global transactions	Horizontal scaling	The 2 M events / s are written to 20 nodes in parallel, with no shared lock
A **standard language** (SQL)	A model fitted to the problem	Friends of friends of friends in 3 lines of Cypher (Neo4j) instead of 3 self-joins

THE FAMILIES

4 historical, 2 new

Key-value

A giant dictionary. Ultra fast, zero complex queries.

`Redis`

Document

Nested JSON objects, queryable by field.

`MongoDB`

Wide-column

Rows with millions of columns, partitioned by key.

`Cassandra`

Graph

Nodes + edges. Traverse instead of join.

`Neo4j`

Search

Inverted index, full text, relevance.

`Elasticsearch`

Vectors

Semantic similarity, RAG, "show me videos like this one".

`Qdrant`

THE REAL MINDSET CHANGE

Schema-on-write vs schema-on-read

Relational: schema-on-write

The database **rejects** what doesn't fit the columns. The contract is checked on write.

The nightclub bouncer: no sneakers, no entry.

Document: schema-on-read

The database **accepts** pretty much anything. The code that reads has to handle the variants.

The festival: everyone gets in, you deal with whatever you find in the tent.

Flexible ≠ schemaless. The schema still exists: it just moved **into your code**. And if nobody documents it, it lives in the head of the person who quit.

SCALE

Vertical vs horizontal

Vertical: a bigger server

More RAM, more CPU. Simple. Until the day the biggest server in the catalogue isn't enough, or costs a kidney.

One Renault 4 with a Ferrari engine.

Horizontal: more servers

Data is split (sharding) across N commodity machines. NoSQL systems were born for this.

A swarm of scooters. Each one carries a piece.

The price: **joins across machines** become impossible or very expensive. Hence "what is read together is stored together".

Boundaries move

- PostgreSQL does indexed **JSONB**, vectors (`pgvector`), geo. "Just use Postgres" has become a meme in its own right.
- MongoDB does **multi-document transactions** since 4.0, full-text search, vectors (Atlas Vector Search).
- Redis does JSON, graphs, time series through modules.
- Conclusion: we no longer choose "SQL or NoSQL". We choose **a primary data model** and accept compromises on the rest.

04 / MONGODB

MongoDB, top-down

Cluster → server → database → collection → document → field

ID CARD

MongoDB

- **2009**, company 10gen (now MongoDB Inc.)
- The name comes from humongous
- Model: **document**, stored as BSON (binary JSON)
- Language: not SQL, a query API made of JSON objects
- SSPL licence (not quite open source), cloud version **Atlas**
- Version used in this course: **7.0**

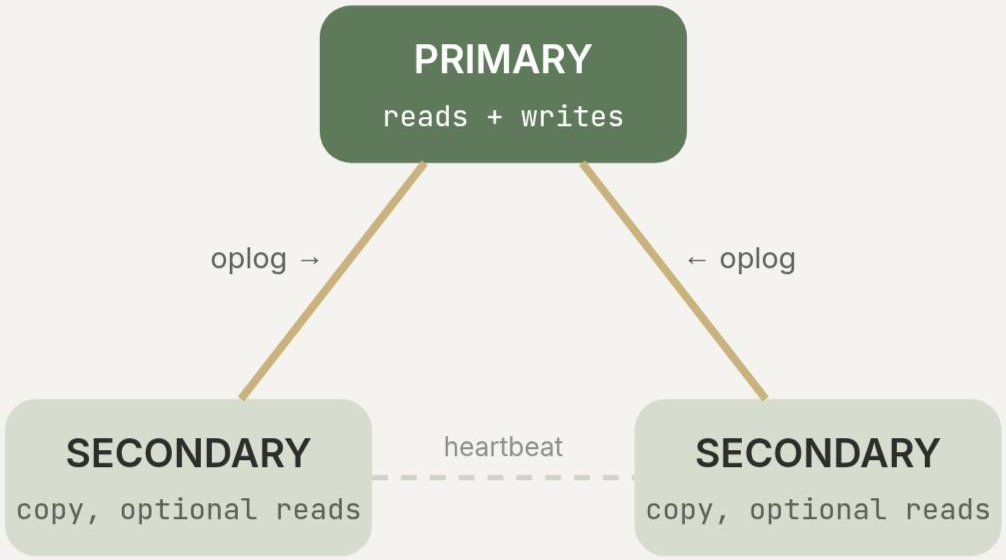
What it does well

Flexible schema, whole-object reads, native horizontal scaling, built-in replication, rich indexes (text, geo, TTL).

What it does less well

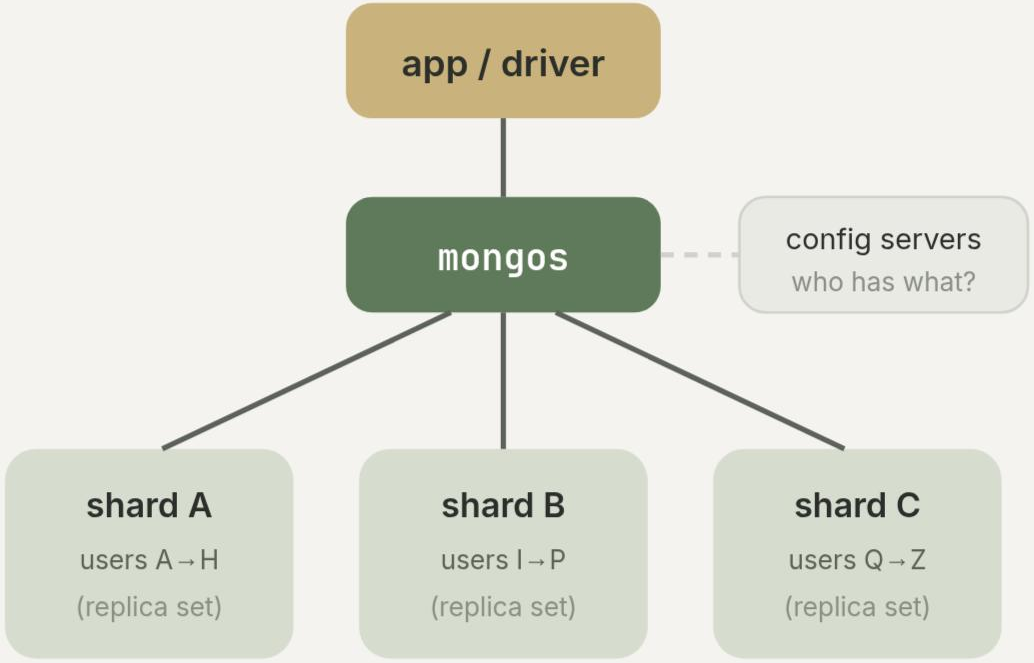
Joins, complex multi-entity transactions, relational reporting.

Replica set: high availability



- Minimum **3 nodes**. One primary receives the writes, the secondaries replay its journal (the oplog).
- Primary goes down? An **election** in a few seconds, a secondary takes over. The app (almost) doesn't notice.
- You can read from secondaries, at the cost of a slight lag. That's the **C vs A** from earlier, tunable per query.

Sharding: scaling out



- Data is **split by a shard key** (e.g. `auteur`) across several replica sets.
- `mongos` is the router: thanks to the config servers, it knows which shard to query.
- Good shard key = queries that hit **one shard**. Bad key = every query everywhere = slower than before.
- For this course: **a single mongod**. Sharding is for when you have 50 M users.

Built in, but not automatic

What comes in the box

Replica sets and sharding **are part of MongoDB**, Community edition included. No extension, no plugin, no third-party software: it is the same `mongod` binary.

What doesn't happen by itself

A `mongod` starts **alone (standalone)**, like in our Docker. You start 3 processes and declare them with `rs.initiate()`, then add `mongos`, the config servers and `sh.shardCollection()` to shard.

- Once configured, the rest is **automatic**: electing a new primary, replication through the oplog, balancing chunks between shards.
- **Atlas**, MongoDB's cloud, does that configuration for you: a "3 nodes" slider, a "shard" button. Paid, but that's where most teams end up.
- In short: by default, a single server. By design, a cluster three commands away.

mongod, mongosh, drivers

`mongod`

The **server**. One process, one port (27017), one data directory. Storage engine **WiredTiger**: compression, in-RAM cache, journal for durability.

`mongosh`

The **shell**. A JavaScript REPL connected to the server. That's what we use today. `db` is the current database.

Drivers

What your **application** uses: `pymongo` (Python), `mongodb` (Node), `mongoose` (Node ODM). Same API as the shell, give or take.

And **Compass**: the official GUI. Handy for exploring, but learn the shell first, that's what ends up in the code.

Server → database → collection → document → field



MongoDB	≈ SQL	But
Server	Server	
Database	Database / schema	created on first write
Collection	Table	**no column contract**
Document	Row	nested, arrays
Field	Column	may be missing, may change type
Index	Index	same, + text, geo, TTL
`$lookup`	JOIN	to avoid

JSON on screen, BSON on disk

```
{
  _id: ObjectId('000000000000000000003e9'),
  auteur: ObjectId('000000000000000000002b'),
  titre: "Micro-trottoir : c'est quoi NoSQL ?",
  duree: 105,
  vues: 1958516,
  hashtags: [ '#lego', '#fitcheck', '#recette' ],
  son: { titre: 'Night Shift', artiste: 'DJ Mongo' },
  publieLe: ISODate('2026-05-28T16:00:00.000Z'),
  produit: { nom: 'Hoodie oversize', prix: 59.99 }
}
```

BSON = Binary JSON. Same shape, but with types JSON doesn't have:

- `ObjectId`, `Date`, `Int32` / `Long`, `Decimal128` (for money!), `Binary`
- Arrays, sub-documents, as many as you like (100 levels max)

Max document size: **16 MB**. If you hit it, your model is wrong, not the limit.

The dataset's field names are in French (`titre`, `duree`, `vues` …): the lab data is shared with the French edition.

The `_id` and the ObjectId

Every document has a **unique, mandatory** `_id`. If you don't provide one, Mongo generates an `ObjectId`: 12 bytes.

```
6ae7c7af 388928108d 8d6ae5
   │          │        └─ counter (3 bytes)
   │          └────────── per-process random (5 bytes)
   └─────────────────────── Unix timestamp (4 bytes)
```

So: **sorted by creation date**, and generatable client-side without asking the server.

In mongosh

```
ObjectId('6ae7c7af388928108d8d6ae5').getTimestamp()
// ISODate('2026-11-02T...')
```

In the course seed, the `_id` s are forced (`…0001` , `…03e9`) so everyone has the same ones. In real life, let Mongo do it.

THE REAL MODELLING QUESTION

Embed or reference?

Embed (sub-document)

```
{ titre: "…", son: { titre: "Tuile", artiste: "Aya N." },
  sondage: { options: [ {texte:"Oui", votes: 4927} ] } }
```

Read together, modified together, bounded size → **embed**.

Reference (ObjectId)

```
{ titre: "…", auteur: ObjectId('…2b') }
// users: { _id: ObjectId('…2b'), pseudo: "jules.vlog" }
```

Shared by many, grows without bound, modified independently → **reference**.

> Golden rule: **what is read together is stored together.** We model for reads, not for purity.

The comments of a viral video: 2 million. Embedded in the video, we blow past 16 MB. So: referenced.

INDEXES

Without an index, every query reads the whole collection

```
db.videos.createIndex({ hashtags: 1 })                // multikey (array)
db.videos.createIndex({ auteur: 1, publieLe: -1 })    // compound
db.users.createIndex({ pseudo: 1 }, { unique: true })
db.videos.createIndex({ titre: "text" })              // full text
db.videos.createIndex({ lieu: "2dsphere" })           // geo
db.sessions.createIndex({ createdAt: 1 }, { expireAfterSeconds: 3600 }) // TTL

db.videos.find({ hashtags: "#nosql" }).explain("executionStats")
   .executionStats.totalDocsExamined   // → 31   (index used)
db.videos.find({ duree: 42 }).explain("executionStats")
   .executionStats.totalDocsExamined   // → 200  (full scan)
```

- `1` ascending, `-1` descending
- An index on an array indexes **every element**
- Max **64 indexes** per collection, and every index slows down writes
- `explain()` is your best friend. `COLLSCAN` is your enemy.

05 / HANDS-ON

Hands in the JSON

One laptop per group. Every slide: the command, then the real output.

SETUP · 3 MINUTES

Docker, and that's it

MONGOSH

```
cd lab
docker compose up -d                  # mongod + mongo-express (http://localhost:8081, admin / zwipe)
docker exec -it zwipe-mongo mongosh zwipe
```

OUTPUT

```
Current Mongosh Log ID: …
Connecting to: mongodb://127.0.0.1:27017/zwipe?directConnection=true
Using MongoDB: 7.0.x
zwipe>
```

The prompt shows the current database. `db` is now the `zwipe` database… which doesn't exist yet. It will exist on the first write.

Everything is created on demand

MONGOSH

```
show dbs                        // admin, config, local… no zwipe
use zwipe                       // switched to db zwipe
db.createCollection("tests")  // explicit creation (rare)
db.brouillons.insertOne({ x: 1 })  // implicit creation: the usual way
show collections
db.brouillons.drop()
```

OUTPUT

```
{ ok: 1 }
{ acknowledged: true, insertedId: ObjectId('6ae7…') }
brouillons
tests
true
```

Careful: a typo in a collection name silently creates a new collection.

THE DATASET

Load Zwipe

MONGOSH

```
# from your terminal (not inside mongosh)
docker exec zwipe-mongo mongosh --quiet zwipe --file /seed.js
```

OUTPUT

```
✅ Base 'zwipe' prête :
   users    : 50
   videos   : 200
   comments : 500
   events   : 2000
```

users

pseudo (handle), pays (country), age?, bio?, premium?, abonnements[] (follows), badges[]

videos

titre, duree, vues (views), likes, hashtags[], son{} (sound), + duo? lieu? (place) sondage? (poll) produit? sousTitres? filtre? epingle? (pinned)

comments / events

comments: video, auteur, texte, reponses[]? (replies)
events: type, video, user, ts, dureeVue? (watch time)

READ

`find`, `findOne`, projection

MONGOSH

```
db.videos.findOne()
db.videos.find()                          // first 20, "it" for more
db.videos.find({}, { titre: 1, vues: 1, _id: 0 }).limit(3)
```

OUTPUT

```
[
  { titre: "Micro-trottoir : c'est quoi NoSQL ?", vues: 1958516 },
  { titre: 'Ce son va rester dans ta tête', vues: 1550432 },
  { titre: "Storytime : mon prof a dit 'jointure'", vues: 1822014 }
]
```

`find(filter, projection)`

- Empty filter `{}` = everything
- Projection: `1` keeps, `0` excludes. Don't mix, except for `_id`
- `find` returns a **cursor**, not an array. `.toArray()` if needed

SQL equivalent: `SELECT titre, vues FROM videos LIMIT 3`

FILTER · COMPARISON

$eq $ne $gt $gte $lt $lte $in $nin

MONGOSH

```
db.videos.countDocuments({ vues: { $gt: 1000000 } })                  // → 110
db.videos.find({ duree: 42 })                                          // implicit equality
db.users.find({ pays: { $in: ["CA", "SN"] } }, { pseudo: 1, pays: 1, _id: 0 }).limit(4)
db.videos.find({ publieLe: { $gte: ISODate("2026-10-01") } })          // dates compare
db.users.find({ age: { $nin: [18, 19, 20] } })                         // ⚠ includes those WITHOUT age
```

OUTPUT

```
[
  { pseudo: 'noah_2k8', pays: 'CA' },
  { pseudo: 'lucas.gg', pays: 'SN' },
  { pseudo: 'manon.study', pays: 'CA' },
  { pseudo: 'jade.pastel', pays: 'SN' }
]
```

Trap: `$ne` and `$nin` also match documents where the field doesn't exist. Absence counts as "not equal".

FILTER · LOGIC

$and $or $not $nor

MONGOSH

```
// implicit AND: two keys in the same object
db.videos.find({ duree: { $lt: 30 }, vues: { $gt: 1000000 } })

db.videos.countDocuments({ $or: [ { duree: { $lt: 10 } }, { vues: { $gt: 2000000 } } ] })   // → 43
db.videos.countDocuments({ duree: { $not: { $gt: 30 } } })                                  // → 29
db.users.countDocuments({ $nor: [ { age: { $exists: true } }, { bio: { $exists: true } } ] }) // → 4

// explicit $and: required when the same key appears twice
db.videos.find({ $and: [ { hashtags: "#nosql" }, { hashtags: "#humour" } ] })
```

In JSON, the same key twice = the second overwrites the first. Hence the explicit `$and` in that case.

$exists, $type

MONGOSH

```
db.videos.countDocuments({ sondage: { $exists: true } })          // → 35  videos with a poll
db.users.countDocuments({ age: { $exists: false } })              // → 9   minors or shy
db.videos.countDocuments({ "son.artiste": { $type: "null" } })    // → 26  "original sound"
db.videos.find({ duree: { $type: "string" } })                    // → []  (phew)
```

OUTPUT

```
// which fields exist, and in how many documents?
db.videos.aggregate([
  { $project: { champs: { $objectToArray: "$$ROOT" } } }, { $unwind: "$champs" },
  { $group: { _id: "$champs.k", n: { $sum: 1 } } }, { $sort: { n: -1 } } ])
// titre, duree, vues… : 200   sousTitres: 62   lieu: 49   produit: 40
// duo: 36   sondage: 35   filtre: 17   epingle: 12
```

This is **exactly U2**. Every video has its own fields. No `ALTER TABLE`, no NULL.

$all $elemMatch $size and dot notation

MONGOSH

```
db.videos.find({ hashtags: "#nosql" })                          // contains (not equals) "#nosql"
db.videos.find({ hashtags: { $all: ["#nosql", "#humour"] } })   // contains both   → 3 videos
db.videos.countDocuments({ hashtags: { $size: 5 } })             // exactly 5 tags  → 41
db.videos.countDocuments({ "son.artiste": "DJ Mongo" })          // nested field    → 24
db.videos.find({ "sondage.options": { $elemMatch: { texte: "Oui", votes: { $gt: 4000 } } } },
               { titre: 1, sondage: 1, _id: 0 }).limit(1)
```

OUTPUT

```
[ { titre: 'Je réponds à vos commentaires',
    sondage: { question: 'Bleu ou rouge ?',
               options: [ { texte: 'Oui', votes: 4927 }, { texte: 'Non', votes: 4293 } ] } } ]
```

`$elemMatch` : one single element must satisfy all conditions. Without it, "texte Oui" and "votes > 4000" could come from two different options.

FILTER · EVALUATION

$regex $expr $mod $text

MONGOSH

```
db.users.find({ pseudo: /^l/ }, { pseudo: 1, _id: 0 })                // native JS regex
db.users.find({ pseudo: { $regex: "scroll", $options: "i" } })
db.videos.countDocuments({ $expr: { $gt: ["$likes", { $multiply: ["$vues", 0.15] }] } }) // → 35
db.videos.countDocuments({ duree: { $mod: [60, 0] } })                // duration in whole minutes → 2
db.videos.find({ $text: { $search: "chat" } }, { titre: 1, _id: 0 }).limit(1)  // text index required
```

OUTPUT

```
[ { pseudo: 'lea.scrolls' }, { pseudo: 'leo.skate' }, { pseudo: 'lina.v' },
  { pseudo: 'lola.paris' }, { pseudo: 'louna.cam' }, { pseudo: 'lucas.gg' } ]
[ { titre: 'Mon chat découvre la 5G' } ]
```

`$expr` compares **two fields of the same document**, impossible otherwise. A regex without `^` can't use an index: full scan.

`sort limit skip` and the `skip` trap

MONGOSH

```
db.videos.find({}, { titre: 1, vues: 1, _id: 0 }).sort({ vues: -1 }).limit(3)
db.videos.find({}, { titre: 1, vues: 1, _id: 0 }).sort({ vues: -1 }).skip(3).limit(2)  // "page 2"
```

OUTPUT

```
[ { titre: 'Bug SQL à 3h du matin', vues: 2491073 },
  { titre: 'Ratio + pas de base de données', vues: 2472275 },
  { titre: 'Je réponds à vos commentaires', vues: 2458286 } ]
[ { titre: 'GRWM avant le partiel', vues: 2417130 },
  { titre: 'Recette en 15 sec (promis)', vues: 2415892 } ]
```

Why an infinite feed never uses `skip`

`skip(1000000)` reads and throws away 1 million documents. Instead, cursor pagination: `find({ vues: { $lt: lastSeenValue } }).sort({ vues: -1 }).limit(20)`. That's U1 too.

COUNT

countDocuments distinct estimatedDocumentCount

MONGOSH

```
db.events.countDocuments({ type: "like" })      // → 252   exact, accepts a filter
db.events.estimatedDocumentCount()              // → 2000  metadata, instant, no filter
db.users.distinct("pays")
```

OUTPUT

```
[ 'BE', 'CA', 'CH', 'DZ', 'FR', 'MA', 'SN', 'TN' ]
```

`count()` still exists but is deprecated. Use `countDocuments` (exact) or `estimatedDocumentCount` (fast).

The pipeline: $match → $group → $sort

MONGOSH

```
db.events.aggregate([
  { $group: { _id: "$type", n: { $sum: 1 }, dureeMoy: { $avg: "$dureeVue" } } },
  { $sort: { n: -1 } }
])
db.videos.aggregate([
  { $unwind: "$hashtags" },
  { $group: { _id: "$hashtags", videos: { $sum: 1 }, vues: { $sum: "$vues" } } },
  { $sort: { vues: -1 } }, { $limit: 3 }
])
```

OUTPUT

```
[ { _id: 'vue', n: 1003, dureeMoy: 30.12 }, { _id: 'skip', n: 512, dureeMoy: 31.24 },
  { _id: 'like', n: 252, dureeMoy: null }, { _id: 'replay', n: 233, dureeMoy: null } ]
[ { _id: '#etudiant', videos: 39, vues: 53289904 }, { _id: '#nosql', videos: 31, vues: 45277793 },
  { _id: '#sneakers', videos: 37, vues: 45260682 } ]
```

Each stage receives the documents of the previous one. It is Mongo's `GROUP BY`, only more powerful. Session 2: we spend an hour on it.

AGGREGATE · THE JOIN YOU WILL REGRET

$lookup

MONGOSH

```
db.videos.aggregate([
  { $match: { epingle: true } },
  { $lookup: { from: "users", localField: "auteur", foreignField: "_id", as: "auteur" } },
  { $project: { titre: 1, "auteur.pseudo": 1, _id: 0 } },
  { $limit: 2 }
])
```

OUTPUT

```
[ { auteur: [ { pseudo: 'leo.skate' } ], titre: "Storytime : mon prof a dit 'jointure'" },
  { auteur: [ { pseudo: 'jules.vlog' } ], titre: 'Tuto : paraître productif' } ]
```

- It works. The result is an **array** (even for a single match).
- Doesn't work across shards, not indexed like a SQL JOIN, slow on large volumes.
- **If you use `$lookup` everywhere, you wanted a SQL database.** Or your model embeds too little.

WRITE

insertOne, insertMany

MONGOSH

```
db.videos.insertOne({ titre: "Mon premier Zwipe", duree: 12, vues: 0, likes: 0, hashtags: ["#nosql"],
                      son: { titre: "son original", artiste: null }, publieLe: new Date(),
                      auteur: ObjectId("000000000000000000000001") })
db.videos.insertMany([
  { titre: "Vidéo normale", duree: 30, vues: 0, likes: 0, hashtags: ["#test"], publieLe: new Date() },
  { titre: "Vidéo avec un champ inventé", duree: 30, vues: 0, likes: 0, hashtags: [],
    publieLe: new Date(), meteo: "☀️", nombreDeChats: 3 }      // Mongo doesn't blink
])
```

OUTPUT

```
{ acknowledged: true, insertedId: ObjectId('6ae7c7af388928108d8d6ae5') }
{ acknowledged: true,
  insertedIds: { '0': ObjectId('6ae7c7af388928108d8d6ae6'), '1': ObjectId('6ae7c7af388928108d8d6ae7') } }
```

`insertMany(docs, { ordered: false })` : keeps going after an error instead of stopping. Example: duplicate `_id` → `E11000 duplicate key error`, but the others go through.

UPDATE

`updateOne`, `updateMany` and the operators

MONGOSH

```
db.videos.updateOne({ titre: "Mon premier Zwipe" }, { $set: { epingle: true }, $inc: { vues: 1 } })
db.videos.updateMany({ duree: { $lt: 10 } }, { $set: { format: "ultra-court" } })  // matched 8
db.videos.updateOne({ titre: "Mon premier Zwipe" }, { $push: { hashtags: "#pourtoi" } })
db.videos.updateOne({ titre: "Mon premier Zwipe" }, { $addToSet: { hashtags: "#nosql" } })  // already there →
db.videos.updateOne({ titre: "Mon premier Zwipe" }, { $pull: { hashtags: "#pourtoi" } })
db.videos.updateOne({ titre: "Vidéo avec un champ inventé" },
                    { $unset: { meteo: "" }, $rename: { nombreDeChats: "chats" } })
db.users.updateOne({ pseudo: "fantome" }, { $set: { pays: "FR" } }, { upsert: true })  // creates if missing
```

OUTPUT

```
{ acknowledged: true, insertedId: null, matchedCount: 1, modifiedCount: 1, upsertedCount: 0 }
{ acknowledged: true, insertedId: ObjectId('6ae7c7ba…'), matchedCount: 0, modifiedCount: 0,
  upsertedCount: 1 }
```

Without an operator (`$set`…), `updateOne` refuses. `$inc` is atomic: that's U1's view counter, with no lock.

`replaceOne`, `findOneAndUpdate`, and a conflict

MONGOSH

```
db.videos.replaceOne({ titre: "Vidéo normale" }, { titre: "Vidéo remplacée", duree: 1 })
db.videos.findOne({ titre: "Vidéo remplacée" })
// → { _id: ObjectId('…'), titre: 'Vidéo remplacée', duree: 1 }   every other field is gone

db.videos.findOneAndUpdate({ titre: "Mon premier Zwipe" }, { $inc: { likes: 1 } },
                           { returnDocument: "after", projection: { titre: 1, likes: 1, _id: 0 } })
// → { titre: 'Mon premier Zwipe', likes: 1 }   read + write in one atomic operation

db.videos.updateOne({ titre: "Mon premier Zwipe" },
                    { $push: { hashtags: "a" }, $addToSet: { hashtags: "b" } })
// → MongoServerError: Updating the path 'hashtags' would create a conflict at 'hashtags'
```

`replaceOne` replaces **the whole document** except `_id`. Two operators on the same field in one update: rejected.

DELETE

`deleteOne`, `deleteMany`, and no CASCADE

MONGOSH

```
db.videos.deleteOne({ titre: "Vidéo remplacée" })                          // → deletedCount: 1
db.videos.deleteMany({ titre: { $in: ["Mon premier Zwipe", "Vidéo avec un champ inventé"] } })
                                                                            // → deletedCount: 2
db.videos.deleteMany({})          // empties the collection, keeps the indexes
db.videos.drop()                  // removes collection AND indexes

const u = db.users.findOne({ pseudo: "theo_fc" })
db.videos.countDocuments({ auteur: u._id })     // → 6
db.users.deleteOne({ pseudo: "theo_fc" })
db.videos.countDocuments({ auteur: u._id })     // → 6   still there. Orphans.
```

> No foreign keys, so no `ON DELETE CASCADE`. **Referential integrity is you.** In the code, or with a transaction.

To reset the database: `docker exec zwipe-mongo mongosh --quiet zwipe --file /seed.js`

YOUR TURN · 20 MINUTES

Exercises (file `lab/exercices.md`)

Block A · Read

8 filter queries, from easy (`$gt`) to nasty (`$elemMatch` + `$expr`).

Block B · Write

Create your profile, post a video, edit it, delete it cleanly along with its comments.

Block C · Traps

4 queries that return a wrong result with no error. Explain why.

Expected results are in the file (collapsible). Live correction right after.

CORRECTION

We correct together

One group at the keyboard per exercise. The others judge.

Full solutions are in `lab/solutions.md`, available after the session.

06 / DECIDE

When to choose Mongo. When definitely not.

SPECIFICATIONS TO KNOW

The numbers that come up in interviews

Limit	Value
Document size	**16 MB**
Nesting depth	100 levels
Indexes per collection	64
Fields in a compound index	32
Replica set	max 50 members, 7 voting
Default port	27017

Multi-document transactions: since 4.0 (replica set) and 4.2 (sharded). ACID, but slower and limited to 60 s by default. A good model rarely needs them.

writeConcern: how many nodes must confirm a write (`majority` = safe). **readConcern**: how fresh a read must be. It is **CAP tunable per query**.

Schema validation: possible (`$jsonSchema`) and recommended as soon as the team is bigger than 2 people.

CHOOSE MONGO WHEN

Green light

- **The schema evolves fast** and varies from one object to the next: product catalogue, video sheet, profile, CMS, forms.
- **You read whole objects**, rarely cross-cut pieces: "give me this video with everything in it".
- **You must scale horizontally** without rewriting the app: built-in sharding, built-in replication.
- **Fast prototype**: no migration for every idea. The hackathon, the startup, the MVP.
- **Semi-structured data**: application logs, light IoT events, JSON API imports.
- The team codes in JS/Python and thinks **in objects**, not in tables.

DEFINITELY NOT WHEN

Red light

- **The data is strongly relational**: accounting, ERP, invoicing, inventory. Many entities, many links, many cross-cutting reports.
- **Complex transactions** across several entities on every operation: bank transfer, multi-resource booking.
- **Heavy reporting** with joins galore for finance or management control. They want SQL, give them SQL.
- **Your model needs `$lookup` everywhere.** Clear signal: you normalised a document database. You wanted Postgres.
- **Massive event volume** (U4): Mongo copes, but a columnar or time-series database does it better for 5 times less.
- Cache, counters, leaderboards, sessions (U1, U5): **Redis**, not Mongo.

ANTI-PATTERNS

The 5 ways to get Mongo wrong

1 · The endless array

Embedding every comment in the video. The document grows, passes 16 MB, every read loads 2 million comments.

2 · The 15.9 MB document

"We're still under the limit." No. If you are watching the limit, the model is wrong.

3 · SQL in Mongo

One collection per entity, one `$lookup` per relation. You get the worst of both worlds.

4 · The business `_id`

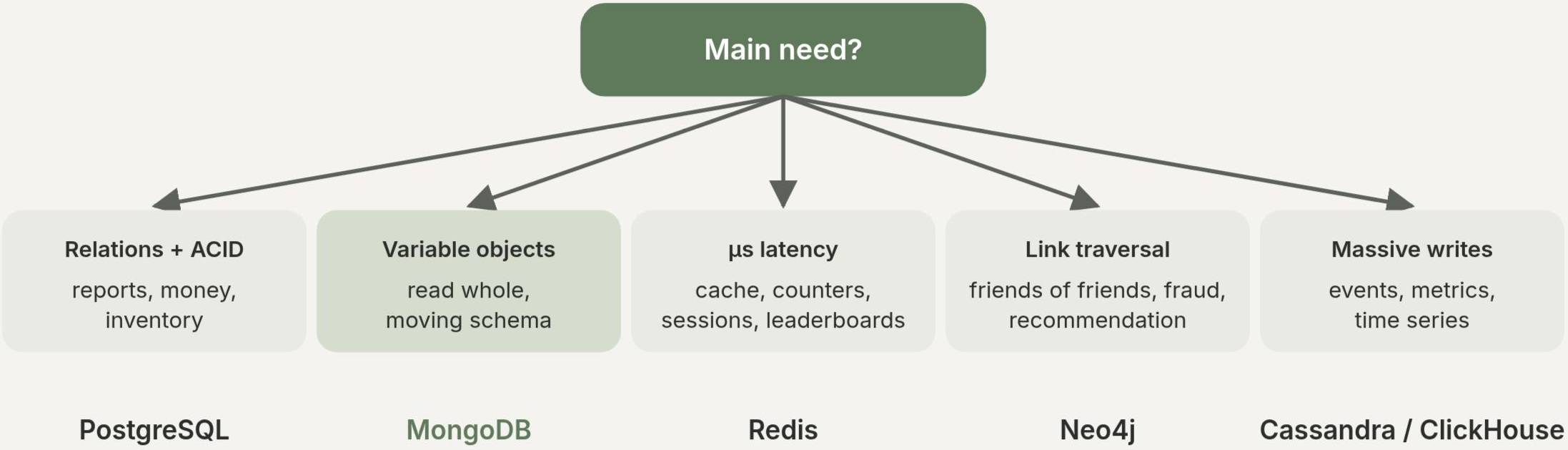
Using the email as `_id`. Until the day the user changes their email. (They will.)

5 · Zero indexes, zero validation

"It's flexible." Flexible doesn't mean "slow and inconsistent". `explain()` and `$jsonSchema` exist.

DECISION TREE

On one slide



"I don't know" → PostgreSQL (with JSONB), then we'll see. You choose per use case, not per application.

Full-text search → Elasticsearch · AI similarity → vector database (or pgvector / Atlas)

The architecture we should have proposed

Case	Technology	Tool	And Mongo?
U1 Feed + counters	In-memory key-value	Redis (`INCR`, lists)	stores the videos behind the cache
U2 Video sheet	Document	**MongoDB**	**yes, that's its job**
U3 Friends of friends	Graph	Neo4j	no (follows as an array is enough for 1 level)
U4 Analytics	Columnar / time series	ClickHouse or Cassandra	possible at small scale, not at 2 M/s
U5 Live	Key-value + pub/sub	Redis (sorted sets, streams)	no
Payments, gifts	Relational	PostgreSQL	no, never, money is ACID

Several technologies for one app: that's "polyglot persistence". As many things to maintain, that's the price.

07 / CLOSING

Quiz, takeaways, what's next

QUIZ · 8 QUESTIONS, ANSWER ON CLICK

Part 1

1. NoSQL means... **Not only SQL**
2. Is MongoDB a tool or a technology? **A tool** (technology: document database)
3. The equivalent of a table in Mongo? **A collection**
4. Max document size? **16 MB**
5. Does `{ age: { $ne: 18 } }` match a document with no `age` field? **Yes.** Absence is not 18.

QUIZ

Part 2

6. `updateOne({…}, { titre: "x" })` without `$set`, what happens? **Error.** An update requires operators. Use `replaceOne` otherwise.
7. The view counter of a viral video: Mongo `$inc` or Redis `INCR`? **Redis** in front, Mongo synced behind. `$inc` is atomic but writes to disk.
8. Why does an infinite feed never use `skip()`? **It reads and discards everything it skips.** Cursor pagination.

TAKEAWAYS

Five sentences

1. SQL didn't fail: it is made for stable relations and transactions, not for everything.
2. You choose a **technology** (data model) before a **tool**.
3. Mongo = JSON documents, flexible schema, **what is read together is stored together**.
4. The schema didn't disappear, it moved into your code. Document it, validate it.
5. A `$lookup` now and then, fine. Everywhere: you wanted Postgres.

SESSION 2

What's next

Advanced aggregation

`$facet`, `$bucket`, windows, `$merge`. Mongo's real analytics engine.

Redis

We build U1 and U5 for real: counters, leaderboard, pub/sub. In 40 lines.

Neo4j

"Your friends liked" in a 3-line Cypher query. And why it is fast.

Until then: finish `lab/exercices.md`, and **model your own app** (one page: collections, what is embedded, what is referenced).

RESOURCES

To dig deeper

- MongoDB documentation: mongodb.com/docs/manual (query operators, data modelling)
- MongoDB University: free courses, "Associate Developer" certification
- Martin Kleppmann, Designing Data-Intensive Applications: the bedside book for this whole course
- db-engines.com/ranking: the database landscape, updated monthly
- This repository: the slides PDF, `lab/`

Thank you. Go scroll.